

Chap 3

Stacks and Queues

海大資訊工程系
蔡東佐



Outline

- Stacks
- Queues
- A Mazing Problem
- Evaluation of Expressions

Stacks

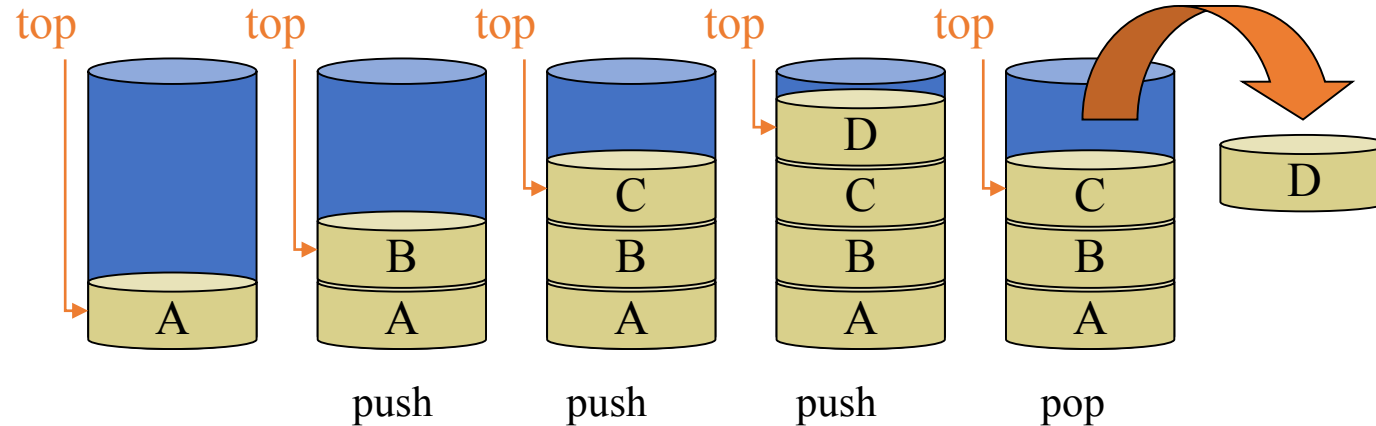
➤ In this chapter we look at two data types that are frequently found in computer science.

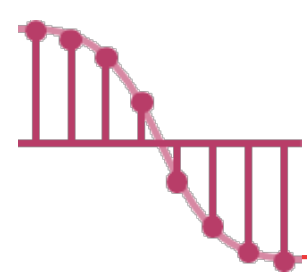
✓ the **stack** and the **queue**

➤ Stack

✓ An ordered list in which **insertions (push/add)** and **deletions (pop/remove)** are made at one end called **top**.

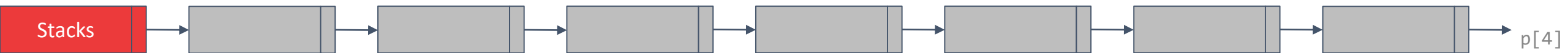
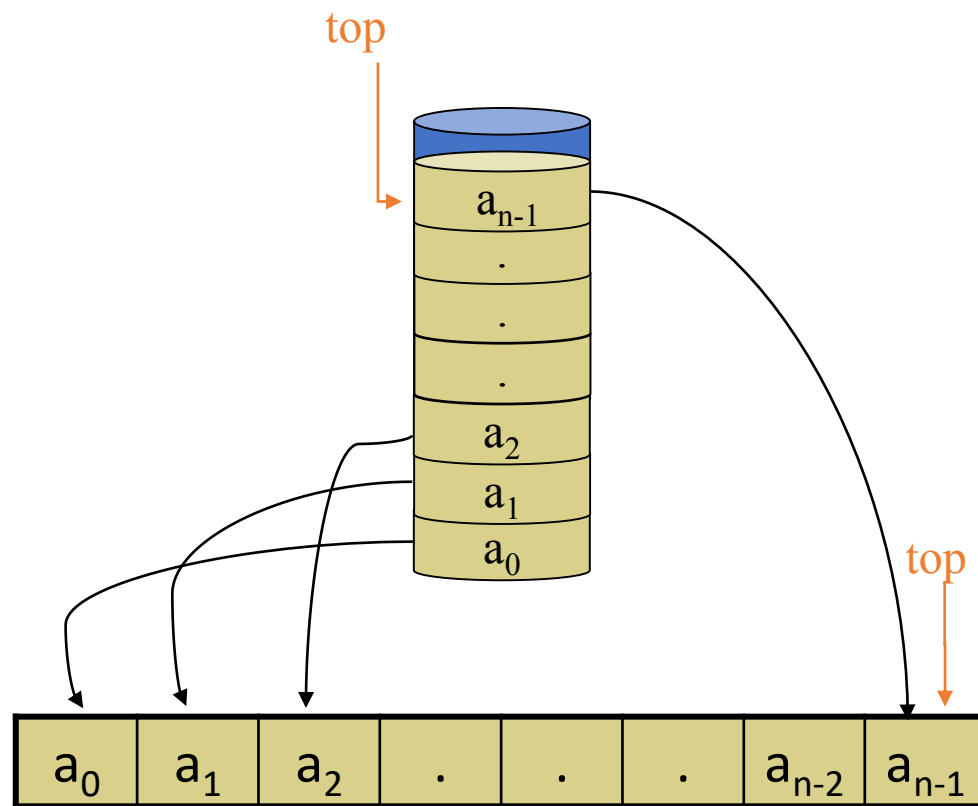
✓ The last element inserted into a stack is the first element removed. **Last-In-First-Out list, LIFO**.





Stacks – **Array** Implementation

- The **easiest way** to implement this ADT is by using a **one-dimensional array**.



Stacks – ADT _{1/2}

➤ We can implement the **operations** in ADT3.1

ADT *Stack* is

物件：一個含有零個或多個元素的有限有序串列

函式：

對所有 $stack \in Stack, item \in element, maxStackSize \in \text{正整數}$

Stack CreateS(**maxStackSize**)::=

// 建立一個容量為 *maxStackSize* 的空堆疊

Boolean IsFull (*stack*, *maxStackSize*)::=

if (*stack* 中的元素個數 == *maxStackSize*)

return *TRUE*

else return *FALSE*

Stack Push(*stack*, *item*)::=

if (IsFull(*stack*)) *stackFull*

else 將 *item* 放在 *stack* 頂端且 return

Boolean IsEmpty (*stack*)::=

if (*stack* == CreateS(*maxStackSize*))

return *TRUE*

else return *FALSE*

Element Pop(*stack*)::=

if (IsEmpty(*stack*)) return

else 移除並回傳堆疊頂端的元素

```
#define MAX_STACK_SIZE 100 /*maximum stack size*/
```

```
typedef struct{
```

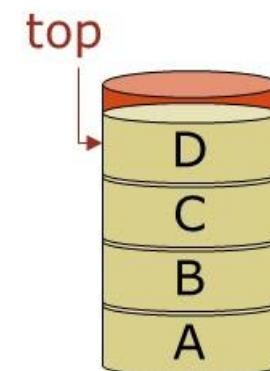
```
    int key;
```

```
    /* other fields */
```

```
} element;
```

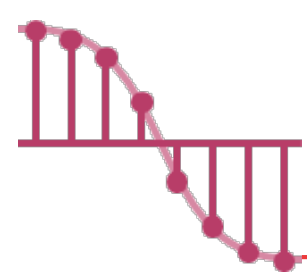
```
element stack[MAX_STACK_SIZE];
```

```
int top = -1;
```



ADT 3.1：抽象資料型態 *Stack*





Stacks – ADT _{2/2}

➤ We can implement the **operations** in ADT3.1

ADT *Stack* is

物件：一個含有零個或多個元素的有限有序串列

函式：

對所有 $stack \in Stack, item \in element, maxStackSize \in \text{正整數}$

Stack CreateS(**maxStackSize**)::=

// 建立一個容量為 *maxStackSize* 的空堆疊

Boolean IsFull (*stack*, *maxStackSize*)::=

if (*stack* 中的元素個數 == *maxStackSize*)

return *TRUE*

else return *FALSE*

Stack Push(*stack*, *item*)::=

if (IsFull(*stack*)) *stackFull*

else 將 *item* 放在 *stack* 頂端且 return

Boolean IsEmpty (*stack*)::=

if (*stack* == CreateS(*maxStackSize*))

return *TRUE*

else return *FALSE*

Element Pop(*stack*)::=

if (IsEmpty(*stack*)) return

else 移除並回傳堆疊頂端的元素

➤ IsEmpty

✓ Return TRUE if stack is empty and FALSE otherwise

top < 0

➤ IsFull

✓ Return TRUE if stack is full and FALSE otherwise

top >= MAX_STACK_SIZE - 1;

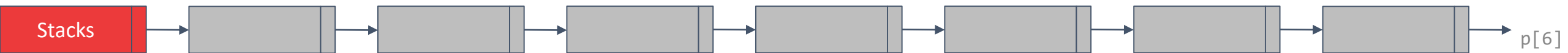
➤ **Add**

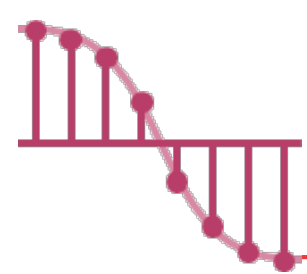
✓ Insert item into top of the stack

➤ **Delete**

✓ Remove and return the item on the top of the stack

ADT 3.1：抽象資料型態 *Stack*



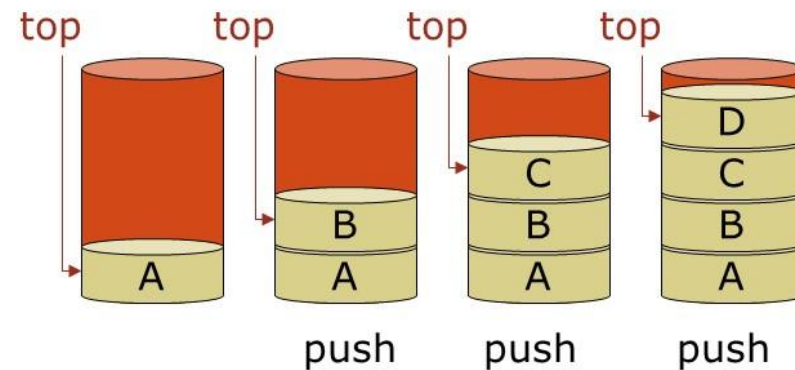


Stacks – Program 3.1

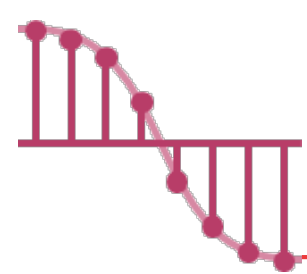
➤ Push (add) to a stack

```
void push (element item)
{
    /* add an item to the global stack */
    if (top >= MAX_STACK_SIZE - 1) {
        stackFull();
    }
    stack[++top] = item;
}
```

top = top + 1



```
void stackFull ()
{
    fprintf(stderr, "Stack is full, cannot add element");
    exit(EXIT_FAILURE);
}
```

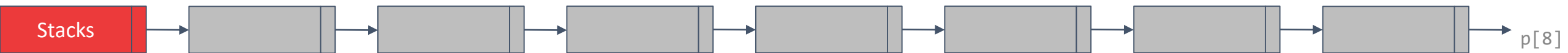
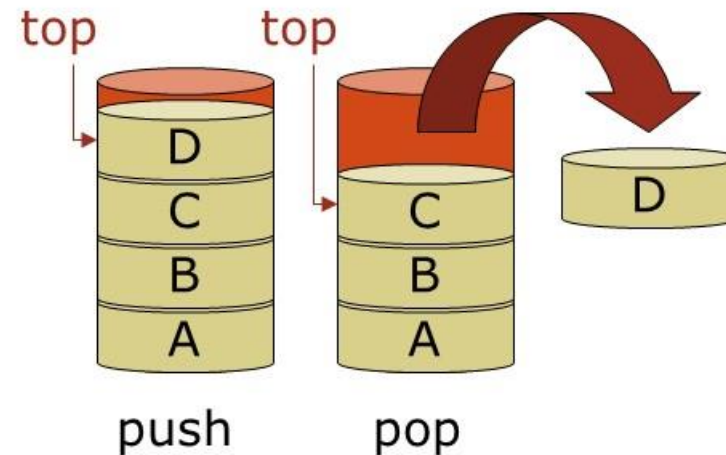


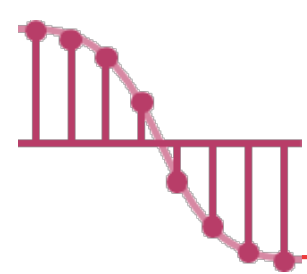
Stacks – Program 3.2

➤ Pop (delete) from a stack

```
element pop ()  
{  
  /* return the top element from the stack */  
  if (top == -1)  
    return stackEmpty(); /* returns an error key */  
  return stack[top--];  
}
```

$\text{top} = \text{top} - 1$

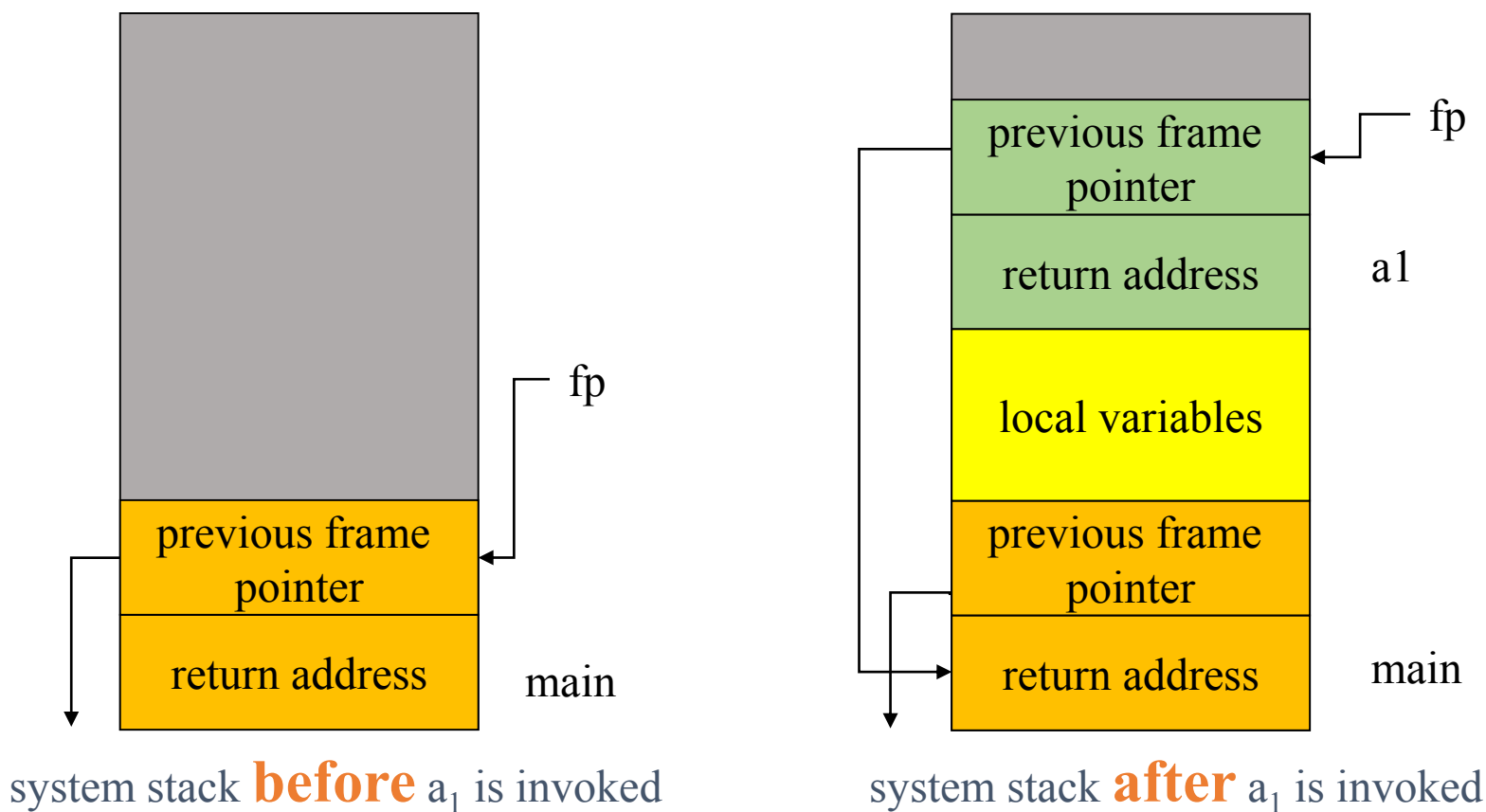




Stacks – Example

C語言函數呼叫在處理區域變數和參數時，就是將這些變數和參數的值都使用堆疊保留下來。程式在呼叫函數前，將返回位址、各區域變數和參數都一一存入堆疊，等到返回後，再一一取出堆疊內容，恢復成函數呼叫前的執行狀態。

➤ System Stack (An Application of Stack: Stack Frame of Function Call)

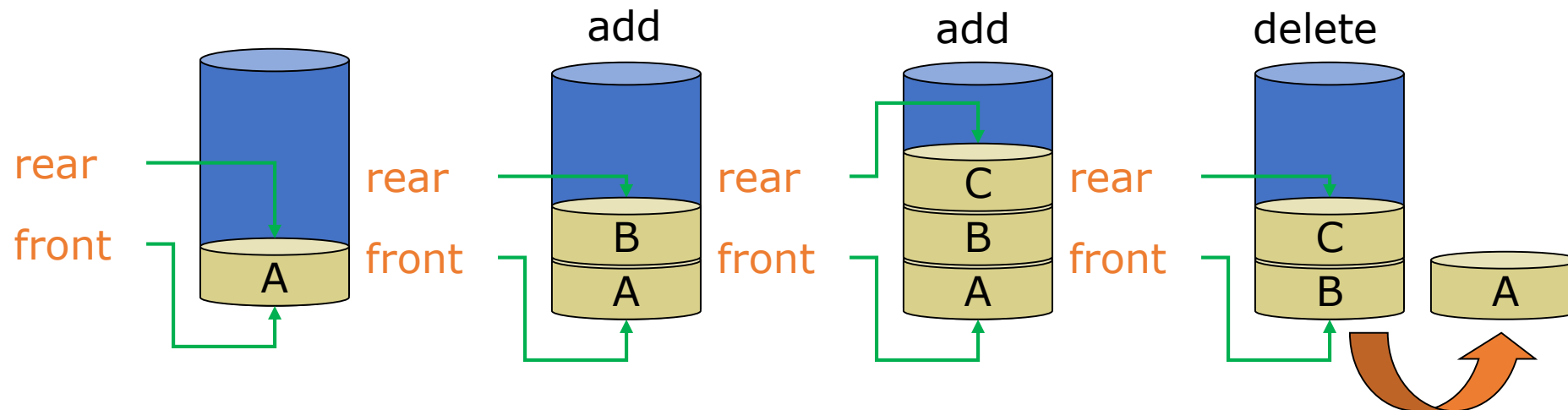


Queues

```
struct {  
    int rear = -1, front = -1;  
    int queue[MAX_QUEUE_SIZE];  
}
```

➤ Queue

- ✓ An ordered list in which **insertions** take place at **one end** and **all deletions** take place at the **opposite end**.
- ✓ The first element inserted into a queue is the first element removed. **First-In-First-Out** list, **FIFO**.



Queues – ADT ^{1/2}

➤ Create Queue (**Array** Implementation)

ADT *Queue* is

物件：一個含有零或多個元素的有限有序串列

函式：對所有 $queue \in Queue$, $item \in element$, $maxQueueSize \in \text{正整數}$

Queue CreateQ($maxQueueSize$)::=

// 建立一個最大尺寸為 $maxQueueSize$ 的空佇列

Boolean IsFullQ ($queue$, $maxQueueSize$)::=

if(佇列中的元素個數 == $maxQueueSize$)

return *TRUE*

else return *FALSE*

Queue AddQ ($queue$, $item$) ::=

if(IsFull($queue$))

else 插入一個項目到 $queue$ 中並回傳 $queue$

Boolean IsEmptyQ ($queue$) ::=

if($queue == \text{CreateQ}(maxQueueSize)$)

return *TRUE*

else return *FALSE*

element DeleteQ($queue$)::=

if(IsEmptyQ ($queue$)) return

else 移除並回傳 $queue$ 前端的項目

```
#define MAX_QUEUE_SIZE 100 /* Maximum queue size */
```

```
typedef struct{
```

```
int key;
```

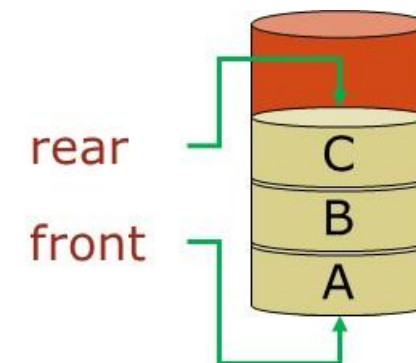
```
/* other fields */
```

```
} element;
```

```
element queue[MAX_QUEUE_SIZE];
```

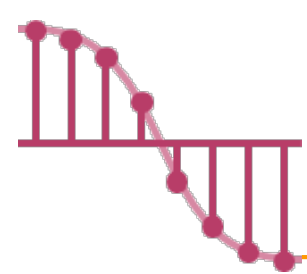
```
int rear = -1;
```

```
int front = -1;
```



ADT 3.2：抽象資料型態 *Queue*





Queues – ADT _{2/2}

+rear;
queue[rear]=v; →

➤ Create Queue (**Array** Implementation)

ADT *Queue* is

物件：一個含有零或多個元素的有限有序串列

函式：對所有 $queue \in Queue$, $item \in element$, $maxQueueSize \in \text{正整數}$

Queue CreateQ($maxQueueSize$)::=

// 建立一個最大尺寸為 $maxQueueSize$ 的空佇列

Boolean IsFullQ($queue$, $maxQueueSize$)::=

if(佇列中的元素個數 == $maxQueueSize$)

return *TRUE*

else return *FALSE*

Queue AddQ($queue$, $item$)::=

if(IsFull($queue$))

else 插入一個項目到 $queue$ 中並回傳 $queue$

Boolean IsEmptyQ($queue$)::=

if($queue == \text{CreateQ}(maxQueueSize)$)

return *TRUE*

else return *FALSE*

element DeleteQ($queue$)::=

if(IsEmptyQ($queue$)) **return**

else 移除並回傳 $queue$ 前端的項目

➤ IsEmptyQ

✓ Return *TRUE* if the queue is empty and *FALSE* otherwise

front == rear;

➤ IsFullQ

✓ Return *TRUE* if the queue is full and *FALSE* otherwise

rear == MAX_QUEUE_SIZE - 1;

➤ AddQ

✓ Insert item at rear of queue and return queue

➤ DeleteQ

✓ Remove and return the item at front of queue

ADT 3.2：抽象資料型態 *Queue*

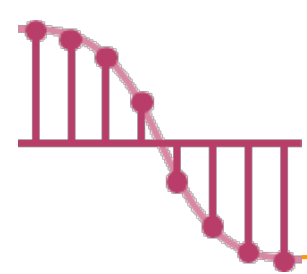




➤ Add to a queue

```
rear = rear + 1
```



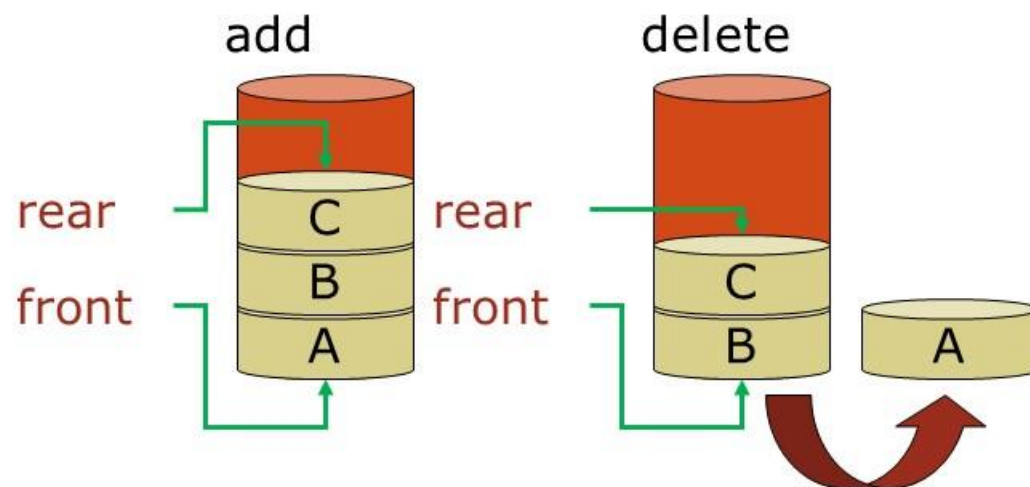


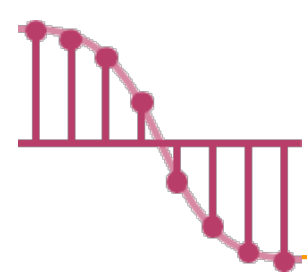
Queues – Program 3.6

➤ Delete from a queue

```
element deleteq ()  
{  
    /* remove element at the front of the queue */  
    if (front == rear)  
        return queueEmpty(); /*return an error key */  
    return queue[++front];  
}
```

$\text{front} = \text{front} + 1$





Queues – Example 3.2 会考

➤ Job scheduling

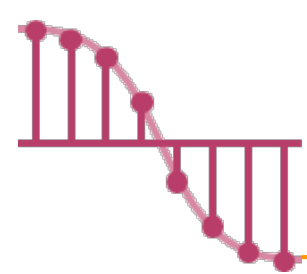
- ✓ Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system.
- ✓ The following Figure illustrates how an operating system might process jobs if it used a sequential representation for its queue.

- The queue gradually shift to right.
- If rear index equals $\text{MAX_QUEUE_SIZE} - 1$, suggesting that the queue is full.
- We should move the entire queue to the left:
 $O(\text{MAX_QUEUE_SIZE})$.

非常耗时！

front	rear	Q[0]	Q[1]	Q[2]	Q[3]	Comments
-1	-1					queue is empty
-1	0	J1				Job 1 is added
-1	1	J1	J2			Job 2 is added
-1	2	J1	J2	J3		Job 3 is added
0	2		J2	J3		Job 1 is deleted
1	2			J3		Job 2 is deleted

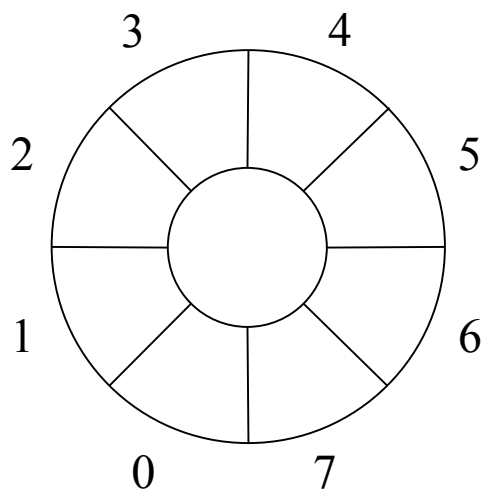




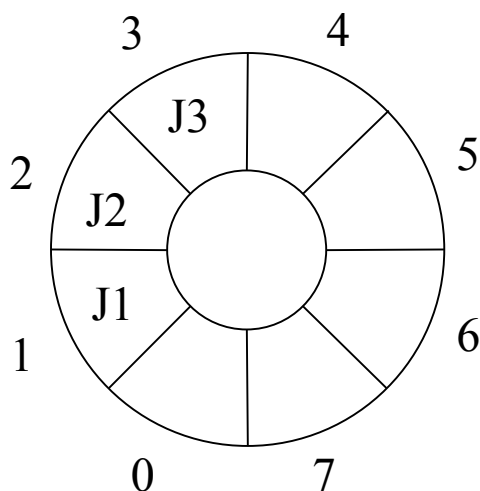
Circular Queues ^{1/2}

➤ Solution: Change straight line array to circle

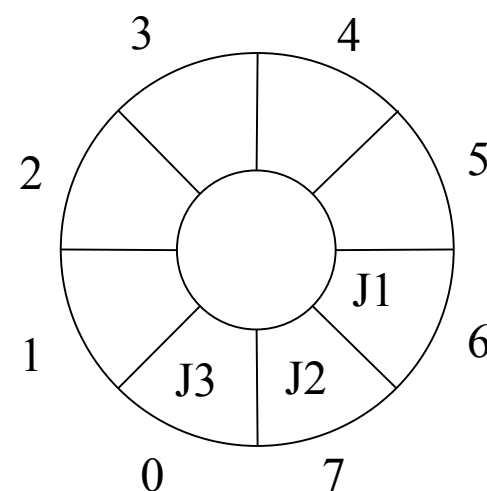
- ✓ Initial circular queue: $\text{front} = \text{rear} = 0$.
- ✓ **front**: one position counterclockwise from the first element in the queue.
- ✓ **rear**: current end of the queue.



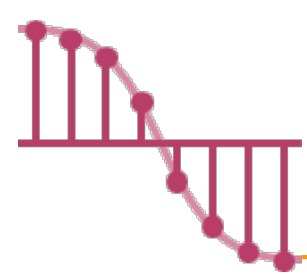
$\text{front} = 0; \text{rear} = 0$



$\text{front} = 0; \text{rear} = 3$

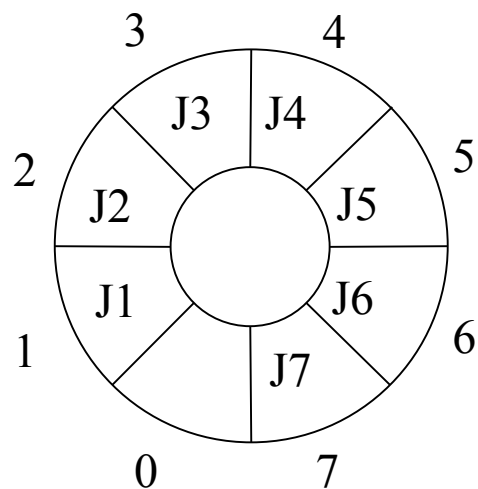


$\text{front} = 5; \text{rear} = 0$

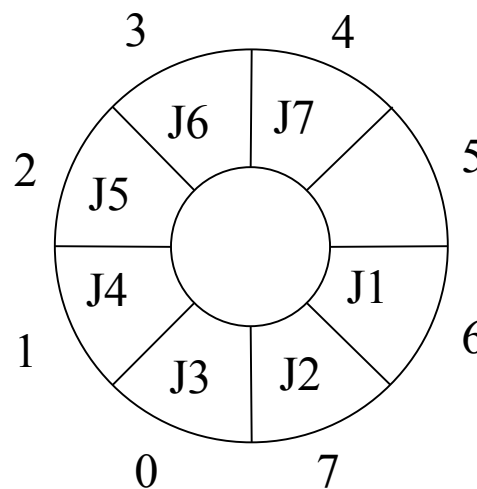


Circular Queues ^{2/2}

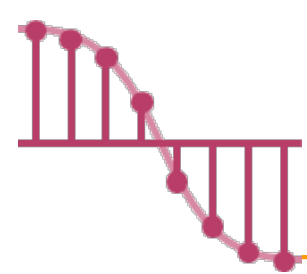
- A circular queue of size **MAX_QUEUE_SIZE** will be permitted to hold **at most MAX_QUEUE_SIZE - 1** elements.
- The addition of such an element will result in **front = rear** and **won't be able to distinguish between an empty and a full queue.**



front = 0; rear = 7



front = 5; rear = 4



Circular Queues – Program 3.7 会考

➤ Add to a circular queue

$$\text{front} = (\text{front} + 1) \% \text{MAX_QUEUE_SIZE}$$

→

```
void addq (element item)
```

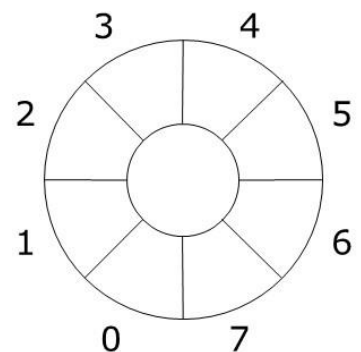
```
{  
    /* add an item to the queue */  
    rear = (rear + 1) % MAX_QUEUE_SIZE;  
    if (front == rear) {  
        queueFull(); /* reset rear and print error */  
    }  
    queue[rear] = item;  
}
```

?: 餘數

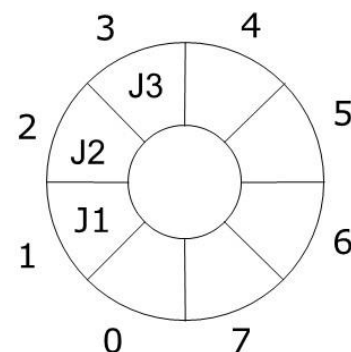
用法: $7 \% 3 = 1$

$8 \% 3 = 2$

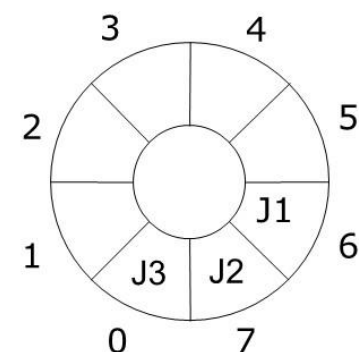
$9 \% 3 = 0$



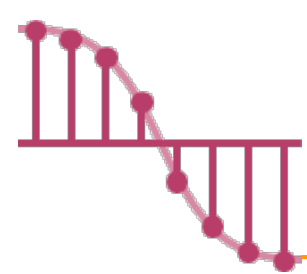
front = 0; rear = 0



front = 0; rear = 3



front = 5; rear = 0

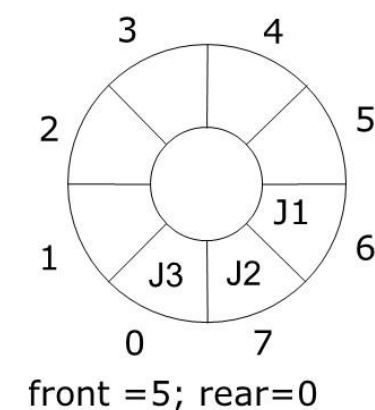
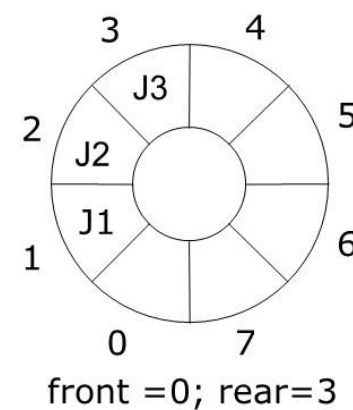
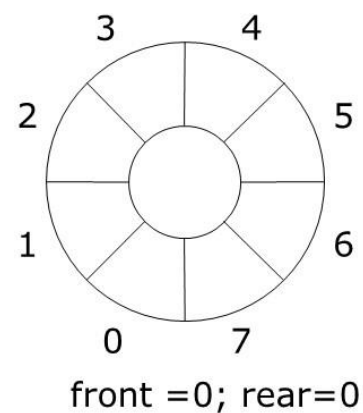


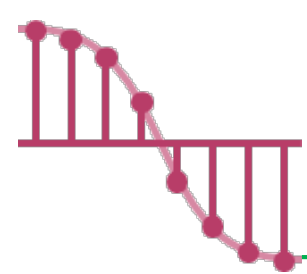
Circular Queues – Program 3.8

会考!

➤ Delete from a circular queue

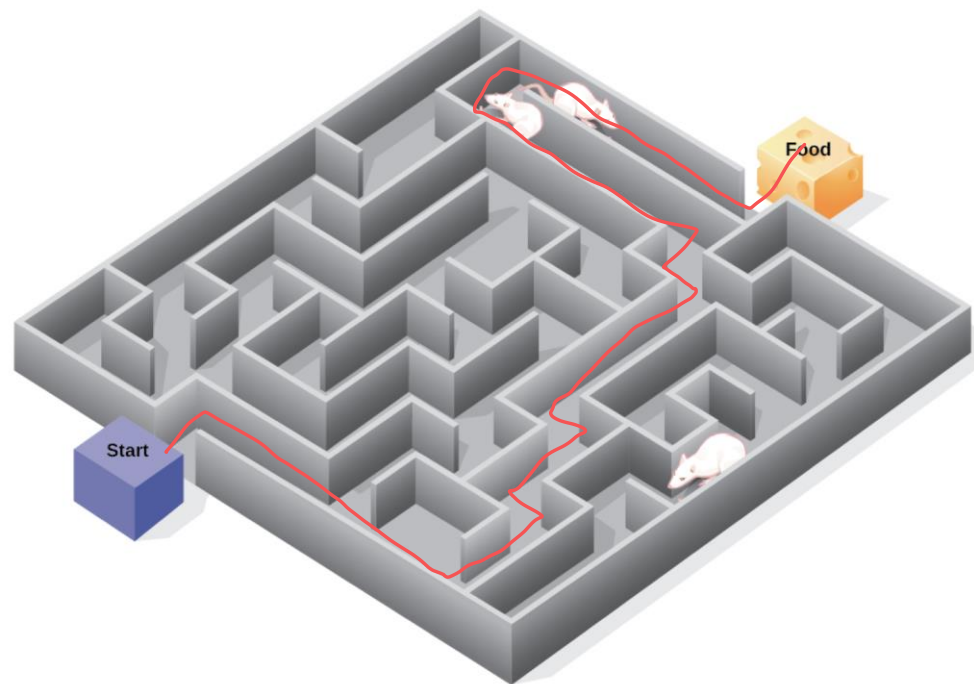
```
element deleteq ()  
{  
    element item; /* remove front element from the queue and put it in item */  
    if (front == rear)  
        return queueEmpty(); /* queueEmpty returns an error key */  
    front = (front + 1) % MAX_QUEUE_SIZE;  
    return queue[front];  
}
```





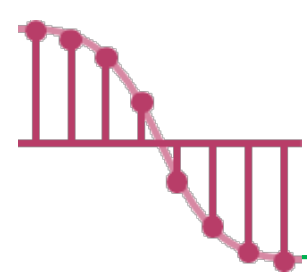
A Mazing Problem ^{1/2}

- Mazes have been an intriguing subject for many years.
 - ✓ Experimental psychologists train rats to search mazes for food



*用 stack
來記錄





A Mazing Problem 2/2

1/5 考!

➤ Example: find the path

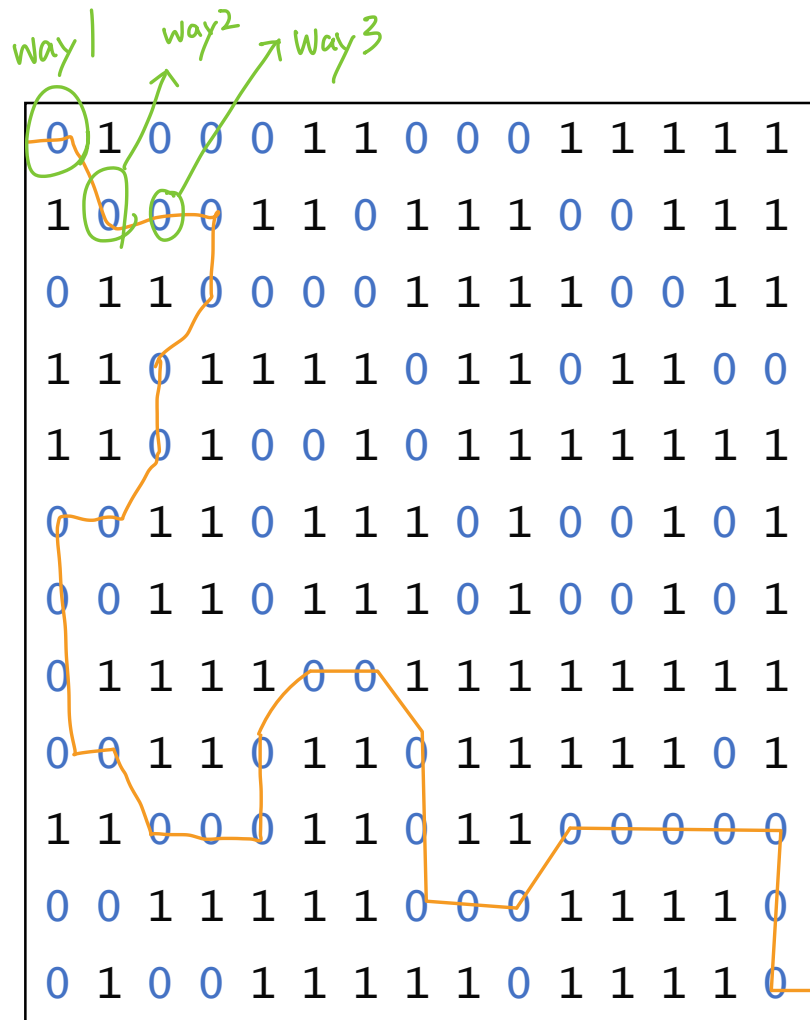
✓ 0: paths

✓ 1: barriers

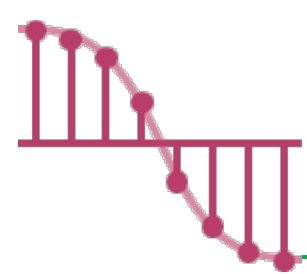
0,0 0,1 0,2
1,0 1,1 1,2
2,0 2,1 2,2

左 上 右 左 左 右 右
上 上 上 0 0 + + +
dx -1 -1 -1 0 0 +1 +1 +1
下 下 下
dy -1 0 +1 -1 +1 -1 0 +1

Entrance



Exit



A Mazing Problem – Next Move

➤ Allowable moves

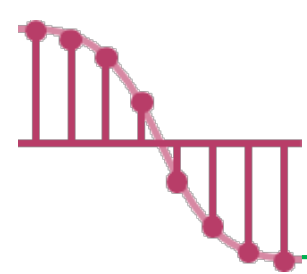
NW [i-1][j-1]	N ② [i-1][j]	NE [i-1][j+1]
W ③ [i][j-1]	X [i][j]	E ① [i][j+1]
SW [i+1][j-1]	S ④ [i+1][j]	SE [i+1][j+1]

```
typedef struct {  
    short int vert;  
    short int horiz;  
} offsets;  
offsets move [8];
```

➤ Table of moves

Name	Dir	Move[dir].vert	Move[dir].horiz
N	0	-1	0
NE	1	-1	1
E	2	0	1
SE	3	1	1
S	4	1	0
SW	5	1	-1
W	6	0	-1
NW	7	-1	-1

```
next_row = row + move[dir].vert;  
next_col = col + move[dir].horiz;
```

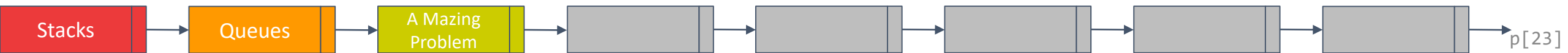


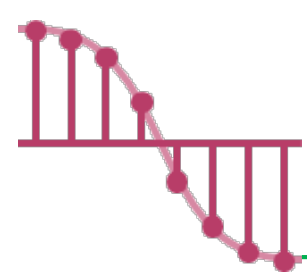
A Mazing Problem – Algorithm _{1/2}

- A second two-dimensional array, **mark**, records the maze positions already checked.
- Use a **stack** to save current path and direction.
 - ✓ We can return to it and try another path if we take a hopeless path.
 - ✓ The stack size is at most $m \times n$, which is the number of positions in the maze.

```
#define MAX_STACK_SIZE 100 /*maximum stack size*/  
typedef struct {  
    short int row;  
    short int col;  
    short int dir;  
} element;  
element stack[MAX_STACK_SIZE];
```

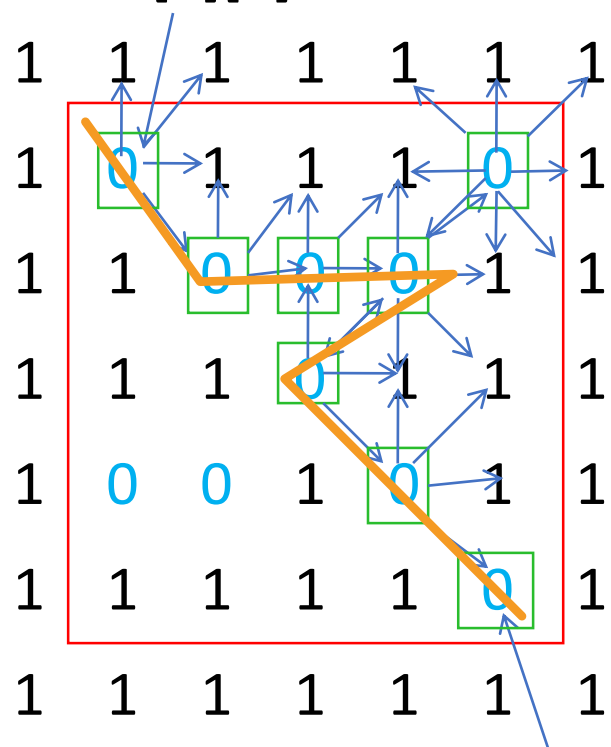
Handwritten notes:
} → now的座標
dir ← 记 move 的 idx





A Mazing Problem – Algorithm _{2/2}

maze[1][1]:entrance



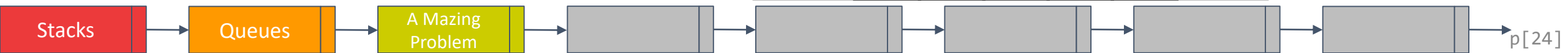
maze[5][5]:exit

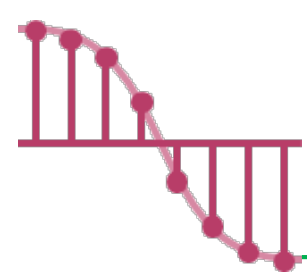
Initially set
mark[1][1]=1

Name	Dir	Move[dir].vert	Move[dir].horiz
N	0	-1	0
NE	1	-1	1
E	2	0	1
SE	3	1	1
S	4	1	0
SW	5	1	-1
W	6	0	-1
NW	7	-1	-1

一定要
好好追 code!
不能直接畫最短路

Row	1	2	2	2	3	4
Col	1	2	3	4	3	4
Dir	0	2	2	1	3	3



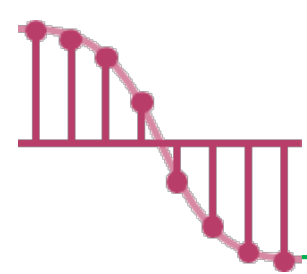


A Mazing Problem – Program 3.11

void path (void)

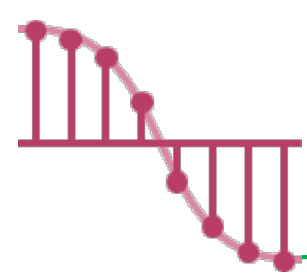
```
{
    /* output a path through the maze if such a path exists */
    int i, row, col, next_row, next_col, dir, found = FALSE;
    element position;
    mark[1][1] = 1; top = 0;
    stack[0].row = 1; stack[0].col = 1; stack[0].dir = 0;
    while (top > -1 && !found) {
        position = delete(&top);
        row = position.row; col = position.col;
        dir = position.dir;
        while (dir < 8 && !found) {
            /*move in direction dir */
            next_row = row + move[dir].vert;
            next_col = col + move[dir].horiz;
            if (next_row==EXIT_ROW && next_col==EXIT_COL)
                found = TRUE;
            else if ( maze[next_row][next_col]==0 &&
                    mark[next_row][next_col]==0){
                mark[next_row][next_col] = 1;

                position.row = row; position.col = col;
                position.dir = ++dir;
                add(&top, position);
                row = next_row; col = next_col; dir = 0;
            }
            else ++dir;
        }
    }
    if (found) {
        printf("The path is :\n");
        printf("row col\n");
        for (i = 0; i <= top; i++)
            printf(" %2d%5d", stack[i].row, stack[i].col);
        printf("%2d%5d\n", row, col);
        printf("%2d%5d\n", EXIT_ROW, EXIT_COL);
    }
    else printf("The maze does not have a path\n");
}
```



Evaluation of Expressions _{1/2}

- The **representation** and **evaluation** of **expressions** is of great interest to computer scientists.
- Example: $((\text{rear} + 1 == \text{front}) \parallel ((\text{rear} == \text{MAX_QUEUE_SIZE} - 1) \&\& !\text{front}))$
 - ✓ operators: $==, +, -, \parallel, \&\&, !$
 - ✓ operands: $\text{rear}, \text{front}, \text{MAX_QUEUE_SIZE}$
 - ✓ parentheses: $(,)$



Evaluation of Expressions _{2/2}

➤ Why expressions are important?

✓ For example

1. $9 + 3 * 5 = ?$

Case a: $9 + 3 * 5 = 24$

Case b: $9 + 3 * 5 = 60$

2. $9 - 3 - 2 = ?$

Case a: $9 - 3 - 2 = 4$

Case b: $9 - 3 - 2 = 8$

✓ The difference between case 1a & case 1b

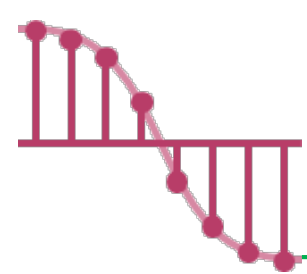
- **Precedence rule**

✓ The difference between case 2a & case 2b

- **Associative rule**

➤ Within **any programming language**, there is a **precedence hierarchy** that determines the order in which we evaluate operators.





Precedence Hierarchy for C

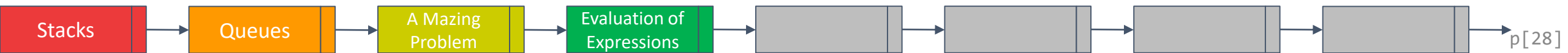
1. The precedence column is taken from Harbison and Steele
2. Postfix form
3. Prefix form

➤ How to generate the machine instructions corresponding to a given expression?

✓ **precedence rule + associative rule**

✓ The **associativity** column indicates how we evaluate operators with the same precedence.

Token	Operator	Precedence ¹	Associativity
() [] →	function call array element struct or union member	17	left-to-right
-- ++	increment, decrement ²	16	left-to-right
-- ++ ! ~ - + & * sizeof	decrement, increment ³ logical not one's complement unary minus or plus address or indirection size (in bytes)	15	right-to- left
(type)	type cast	14	right-to- left
* / %	multiplicative	13	left-to-right
+ -	binary add or subtract	12	left-to-right
<< >>	shift	11	left-to-right
> >= < <=	relational	10	left-to-right
== !=	equality	9	left-to-right
&	bitwise and	8	left-to-right
^	bitwise exclusive or	7	left-to-right
	bitwise or	6	left-to-right
&&	logical and	5	left-to-right
	logical or	4	left-to-right
?:	conditional	3	right-to- left
= += -= /= *= %= <<= >>= &= ^= =	assignment	2	right-to- left
,	comma	1	left-to-right





Evaluating Postfix Expressions ^{1/3}

➤ The **standard way** of writing expressions is known as **infix** notation.

中綴(序)表示法

✓ Place a binary operator in-between its two operands.

• Ex: $a + b$

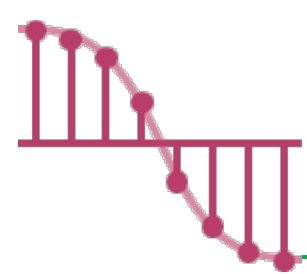
✓ It is **not** the one used by compilers to evaluate expressions.

逆(序)表示法

➤ Instead compilers typically use **postfix**.

Why ?

Infix	Postfix
$2 + 3 * 4$	$234*+$
$a * b + 5$	$ab*5+$
$(1 + 2) * 7$	$12+7*$
$a * b / c$	$ab*c/$
$((a / (b - c + d)) * (e - a) * c$	$abc - d+ / ea - * c *$
$a / b - c + d * e - a * c$	$ab/c - de * + ac * -$



Evaluating Postfix Expressions _{2/3}

➤ Evaluating **postfix** expressions is **much simpler** than the evaluation of infix expressions.

✓ There are **no parentheses and precedence to consider**. (不用考慮原式和優先級!)

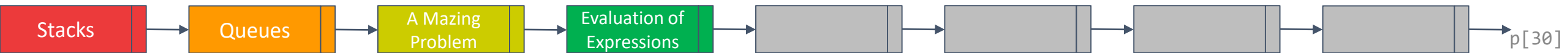
✓ To evaluate an expression we make a single **left-to-right scan** of it.

✓ We can evaluate an expression easily by **using a stack**.

➤ For example, the input is nine character string $6\ 2/3 - 4\ 2*+$

Token	[0]	Stack [1]	[2]	Top
6	6			0
2	6	2		1
/	6/2			0
3	6/2	3		1
-	6/2-3			0
4	6/2-3	4		1
2	6/2-3	4	2	2
*	6/2-3	4*2		1
+	6/2-3+4*2			0

Tips! 看到符號 POP
非符號就 Push





string: 6 2/3-4 2*+ $\Rightarrow 6 \div 2 - 3 + 4 \times 2$

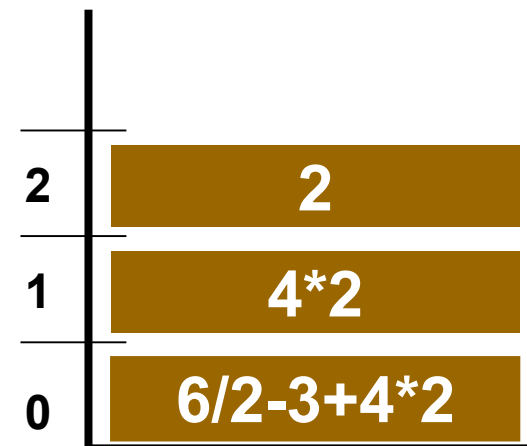
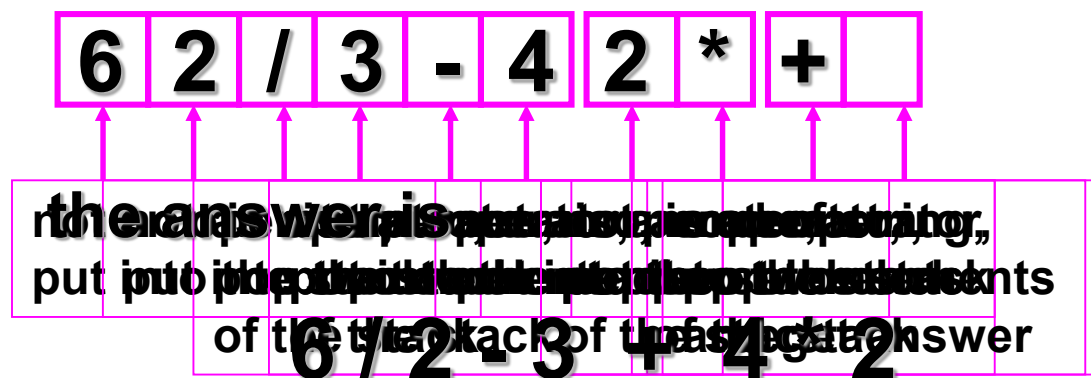
3/3

$((a/b) - c) + (d \times e) - (a \times c)$

$ab/c -$

we make a single left-to-right scan of it

Perform the operator and place the result back on the stack

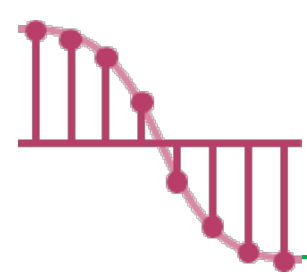


top

STACK

now, top must +2





Infix to Postfix ^{1/4}

➤ Algorithm 1

✓ Ex: $a / b - c + d * e - a * c$

- Step1: fully parenthesize the expression.

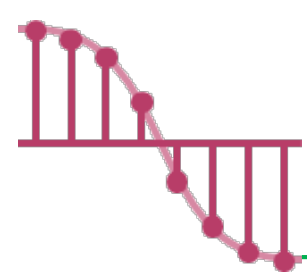
□ $((((a / b) - c) + (d * e)) - (a * c))$

- Step2: move all binary operators so that **they replace their corresponding right parentheses.**

□ $((((ab/c - (de* + ac* -$

- Step3: delete all parentheses.

□ $ab/c - de* + ac* -$



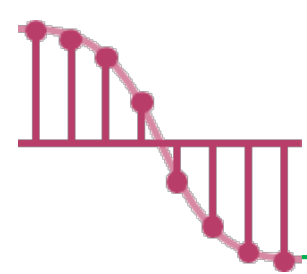
Infix to Postfix 2/4

op	isp	icp
(	0	20
)	19	19
+	12	12
-	12	12
*	13	13
/	13	13
%	13	13
eos	0	0

➤ Algorithm 2

- ✓ Scan string from left to right
- ✓ Operands are **taken out** immediately 遇到運算元 立即輸出
- ✓ Operators are **taken out of the stack** as long as their **in-stack precedence (isp)** is **higher than or equal to** the **incoming precedence (icp)** of the new operator 運算符的isp ≥ 新運算符的icp時馬上將它從stack中pop出
- ✓ If token == right parenthesis, unstack tokens until we reach the corresponding left parenthesis. 遇到")"時，將stack中的運算符pop出，直到遇到相對應的"("

a * (b + c) / d → **a b c + * d /**



Infix to Postfix 3/4

(遇到運算元立即輸出)

(運算符號的 isp 大於等於新的運算符號的 icp 時立即將它自堆疊中推出)

(遇到 “)” 將堆疊中運算符號推出直到遇到相對應的 “(”)

op	isp	icp
(	0	20
)	19	19
+	12	12
-	12	12
*	13	13
/	13	13
%	13	13
eos	0	0

a * (b + c) / d

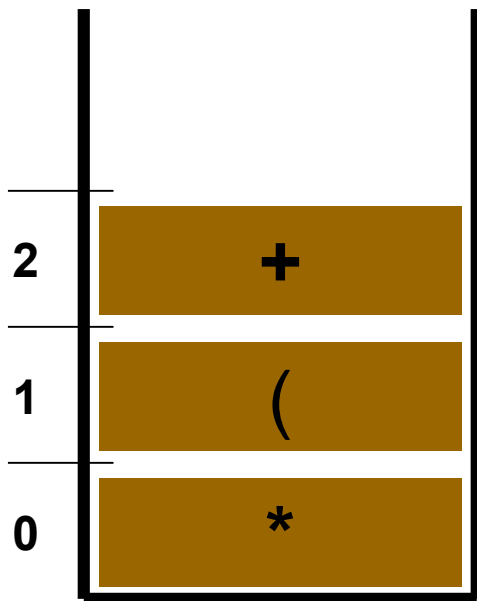
operator precedence table, print out

pop the stack and print out

the output of “(”, “+”, “)” and the isp of “(” is 0

output

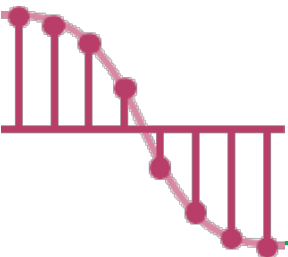
a b c + * d /



top →

stack

now, top must +1



Infix to Postfix 4/4

- (遇到運算元立即輸出)
- (運算符號的 isp 大於等於新的運算符號的 icp 時立即將它自堆疊中推出)
- (遇到 “)” 將堆疊中運算符號推出直到遇到相對應的 “(”)

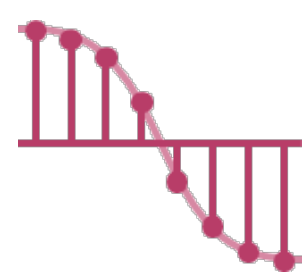
op	isp	icp
(	0	20
)	19	19
+	12	12
-	12	12
*	13	13
/	13	13
%	13	13
eos	0	0

➤ $a + b * c$

Token	Stack [0] [1] [2]	top	output
a		-1	a
+	+	0	a
b	+	0	ab
*	+ *	1	ab
c	+ *	1	abc
eos		-1	abc*+

➤ $a * (b + c) * d$

token	Stack [0] [1] [2]	top	output
a		-1	a
*	*	0	a
(	* (	1	a
b	* (	1	ab
+	* (+	2	ab
c	* (+	2	abc
)	*	0	abc+
*	*	0	abc+*
d	*	0	abc+*d
eos		-1	abc+*d*



```
void postfix(void)
{
    /* output the postfix of the expression. The expression
    string, the stack, and top are global */
    char symbol;
    precedence token;
    int n = 0;
    int top = 0;    /* place eos on stack */
    stack[0] = eos;
    for (token = get_token(&symbol, &n); token != eos;
        token = get_token(&symbol, &n)) {
        if (token == operand)
            printf("%c", symbol);
        else if (token == rparen) {
            /* unstack tokens until left parenthesis */
            while (stack[top] != lparen)
                print_token(delete(&top));
            delete(&top); /* discard the left parenthesis */
        }
        else {
            /* remove and print symbols whose isp is greater
            than or equal to the current token's icp */
            while(isp[stack[top]] >= icp[token])
                print_token(delete(&top));
            add(&top, token);
        }
    }
    while ( (token=delete(&top)) != eos)
        print_token(token);
    printf("\n");
}
```

若為運算元，直接印出

若為右括號

刪除左括號

字串結尾，全部輸出